

KAMARAJ COLLEGE OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ADS

MINI PROJECT REPORT

**ShiftReg: An Interactive Serial Data Transfer Simulator for SISO / SIPO / PISO
/ PIPO Shift Registers**

Submitted by

Veera Lakshmi.B

920425243113

ADS-A

EC2201 & EC2202 – Digital System Design and Microprocessor October
2026

Submission on Time	(5)
Preparation	(10)
Presentation	(10)
Total	

Abstract

Many communication links between digital devices—UART lines, SPI buses, and long sensor cables—provide only a single data wire, even though the data inside a chip is handled as wide parallel words. This project addresses that mismatch by building ShiftReg, a browser-based

simulator that visualizes how shift registers convert between serial and parallel data, one clock edge at a time. The simulator implements all four standard shift-register configurations—Serial-In Serial-Out (SISO), Serial-In Parallel-Out (SIPO), Parallel-In Serial-Out (PISO), and Parallel-In Parallel-Out (PIPO)—using a clock-driven, right-shift logic model in which the input bit is captured at the MSB, all bits shift one place toward the LSB on every rising edge, and the outgoing LSB is emitted as serial output. The tool lets a user select a mode and register width, step or auto-run the clock, and observe the register contents, a live truth table, and an SVG timing diagram update in real time. Correctness is verified with a 15-case validation suite (10 normal and 5 edge/fault cases) generated from a seeded PRNG (mulberry32, seed 42) so every run is reproducible, cross-checking an array-based engine against an independently coded string-based reference model. The result is a self-contained, no-backend educational lab artifact suitable for reinforcing synchronous sequential-circuit concepts taught in EC2201.

1. Introduction

Digital systems are built almost entirely from parallel data paths—buses, registers, and memories that move many bits at once. However, the physical links that connect separate chips or boards are frequently constrained to a single wire per direction, for cost, connector-size, and cabling reasons. Bridging this gap between wide, parallel data and narrow, serial channels is one of the foundational problems in synchronous sequential logic, and it is solved by a circuit block called the shift register. Shift registers underpin common real-world interfaces such as UART, SPI, and long-distance sensor buses, and understanding their clock-by-clock behaviour is essential to digital system design. The objective of this project is to design and implement an interactive, browser-based simulator that visualizes and validates the behaviour of SISO, SIPO, PISO, and PIPO shift registers.

1.1 Real-World Problem / Use Case

Most buses inside a chip are parallel, but many links between chips—SPI, sensor lines, and long cables—only have one data wire. A shift register is the bridge: it converts a wide parallel word into a slow serial stream, or the reverse, one clock edge at a time. This project targets exactly that use case, covering UART links, microcontroller-to-peripheral communication, and SPI/sensor buses as concrete, motivating examples.

2. Problem Statement

The project sets out to build a self-contained, no-backend web application that models and visualizes the four canonical shift-register configurations—SISO, SIPO, PISO, and PIPO—under a single clock-driven, right-shift logic model. On every clock edge, the register must capture the serial input at the MSB, shift every other bit one position toward the LSB, and emit the outgoing LSB as serial output; parallel modes must load or read the entire word in a single step instead of shifting bit-by-bit.

The following assumptions and constraints were adopted:

- Registers power up cleared to 0 (all-zero reset state).
- Shifting is strictly right-shift: MSB-in, LSB-out.
- All inputs are strictly binary (0/1).

- For PISO and PIPO modes, the parallel load word must match the configured register width exactly.
- Selectable register widths are 1, 4, 6, and 8 bits, with 4 bits as the default.

3. Implementation

3.1 Tools and Technologies Used

- HTML5, CSS3, and JavaScript (ES6) — client-side, no backend server required.
- SVG — used to render the live timing diagram (clock, serial-in, and serial-out waveforms).
- Mulberry32 seeded pseudo-random number generator — produces reproducible test vectors (seed = 42) so every run of the validation suite generates identical inputs.
- Netlify — static site hosting for deployment of the finished simulator.
- Browser DevTools / manual testing — used to cross-check the array-based simulation engine against an independently coded string-based reference model.

3.2 Implementation Steps

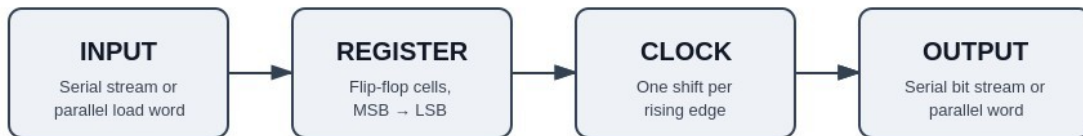
- Step 1 — Define the core shift-register logic model: on each clock edge, capture serial_in at the MSB, shift the internal state array one place toward the LSB, and output the bit that falls off the LSB end.
- Step 2 — Implement the four operating modes (SISO, SIPO, PISO, PIPO) on top of the shared shift-step function, differing only in how data enters (serial feed vs. parallel load) and exits (serial read vs. parallel read).
- Step 3 — Build the control deck UI: mode selector, register-width selector (1/4/6/8 bits), serial input stream field, seed and length controls, and Load/Reset, Step, and Auto-run actions.
- Step 4 — Add a dual-trace register view and a live-updating truth table (Cycle, SI, Register MSB → LSB, SO, Note) so every clock edge is traceable.
- Step 5 — Render an SVG timing diagram plotting clock, serial-in, and serial-out levels across the cycles run so far.

- Step 6 — Build a 15-case validation suite (10 normal + 5 edge/fault cases) driven by a seeded PRNG, cross-checking the array-based engine against a second, independently coded string-based reference model, and surface a pass/fail result and pass rate for each run.
- Step 7 — Package documentation (project objective, assumptions, future improvements) and deployment-ready assets, then deploy the static site to Netlify.

3.3 System Design / Architecture

Regardless of mode, every transfer moves through the same four stages, illustrated in Fig. 1. Only the entry point (serial feed or parallel load) and exit point (serial read or parallel read) differ between SISO, SIPO, PISO, and PIPO.

Fig. 1 — Shift Register Signal Path (SISO / SIPO / PISO / PIPO)



Each rising clock edge: capture input bit at MSB, shift all bits one place toward LSB, emit old LSB as serial output

Internally, each clock edge executes the shift-right update shown below, where state is the bit array held MSB → LSB:

```

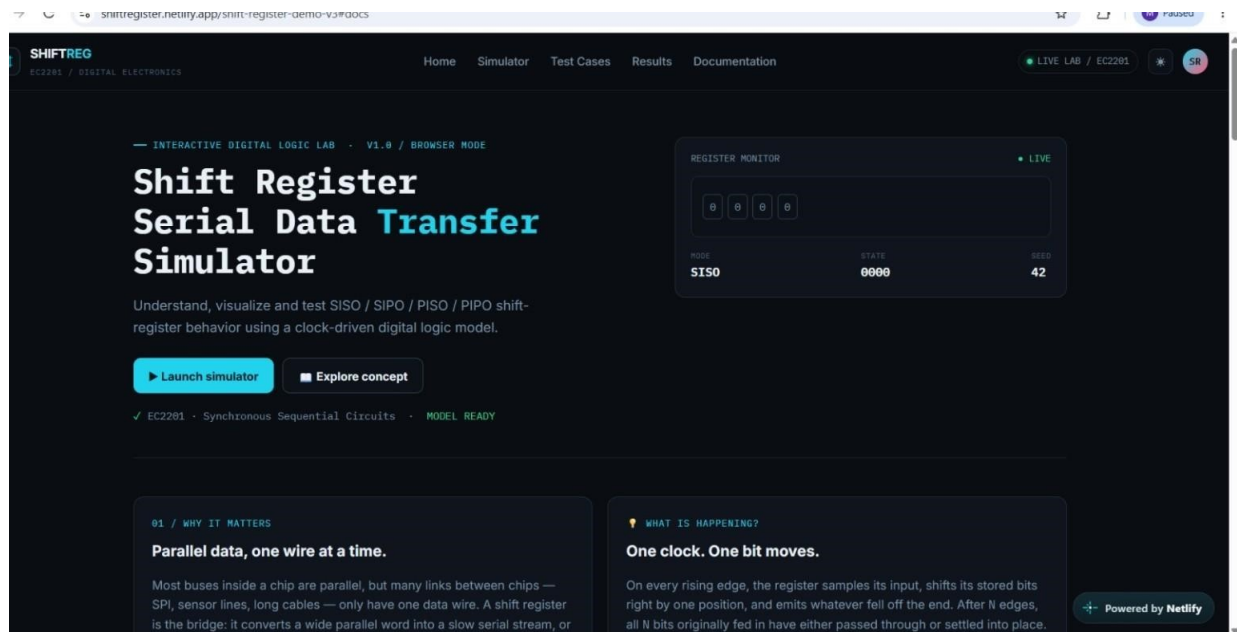
on clock_edge:
    so = state[width-1]
    state = [serial_in] + state[0 : width-1]
  
```

A representative clock-by-clock trace for a 4-bit register, seeded with the input stream 1011..., is shown below.

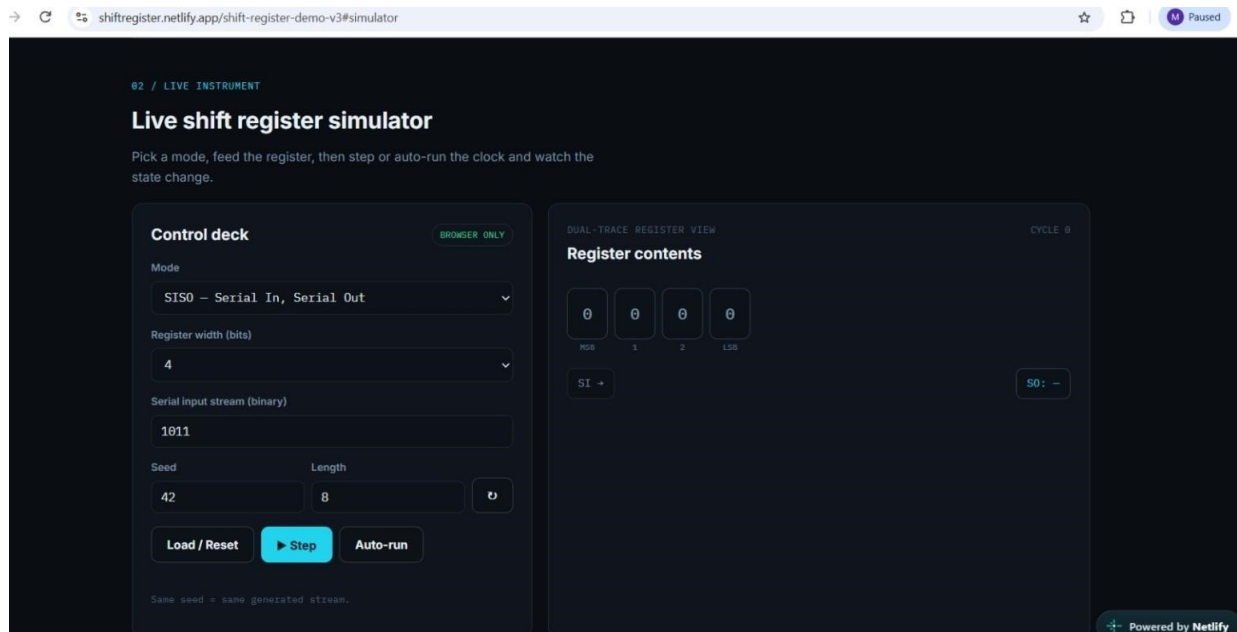
Cycle	SI	Register (MSB → LSB)	SO	Note
0	-	0000	-	Reset state
1	1	1000	0	First bit loaded at MSB
2	0	0100	0	Bit shifts right
3	1	1010	0	New bit enters MSB
4	1	1101	0	Register fills
5	0	0110	1	First input bit exits at LSB

4. Output / Screenshots

The deployed simulator is available at: shiftregister.netlify.app/shift-register-demo-v3. Insert screenshots of the working application below, matching the demo flow: (1) the landing/hero view with the register monitor, (2) the live control deck with a mode selected and a register mid-shift, (3) the truth table and timing diagram after several clock steps, and (4) the validation suite showing 15/15 tests passing. Each screenshot should carry a one-line caption describing what it demonstrates.



Screenshot 1 — Landing view / Register Monitor



Screenshot 2 — Live control deck: mode, width, step/auto-run

[Insert Screenshot 3 — Truth table and SVG timing diagram after N cycles]

[Insert Screenshot 4 — Validation suite results (15/15 tests passing)]

5. Conclusion and Future Scope

ShiftReg successfully demonstrates the behaviour of all four standard shift-register configurations through a single, clock-driven right-shift model, presented as an interactive, nobackend web lab. The live register view, truth table, and SVG timing diagram make the bitby-bit transfer process visible and easy to reason about, while the 15-case seeded validation suite gives confidence that the implementation is correct and its results are reproducible.

A current limitation is that only right-shift behaviour and widths up to 8 bits are supported, and each register operates in isolation. As documented future improvements, the project can be extended to add a left-shift mode, cascade multiple registers together for wider serial links, and introduce a bit-error-rate view to model noisy serial channels.

6. References

- [1] ShiftReg — Serial Data Transfer Simulator. Available: <https://shiftregister.netlify.app/shift-register-demo-v3>
- [2] Author, "Digital Logic and Computer Design", Source, Year.
- [3] Author, "Synchronous Sequential Circuits — Shift Registers", Source, Year.

